
DML – Teil 1

BITLC | Tom Selig
7. August 2026

Aufgaben der DML

- Die DML ist eine der fünf Untergruppen von SQL.
- Sie stellt die C(R)UD-Funktionalitäten bereit um die Daten in einer Datenbank zu manipulieren

Create => Erzeugen von neuen Daten

Read => Lesen der vorhandenen Daten (eigentlich DQL)

Update => Verändern der vorhandenen Daten

Delete => Löschen von bestehenden Daten

- Hier geht es also um die Daten selbst, die sich innerhalb einer fest vorgegebenen Struktur befinden.

Beispiel-Tabellen

- Die folgenden Folien nutzen als Beispiel die Tabellen kunde und bestellung.
- kunde:
id, vorname, nachname, geburt
- bestellung:
id, kunde_id, datum, summe
- Die Skripte zur Erstellung finden Sie rechts =>

```
CREATE TABLE kunde (  
    id INT,  
    vorname VARCHAR(20),  
    nachname VARCHAR(20),  
    geburt DATE,  
    CONSTRAINT pk_kunde  
        PRIMARY KEY (id)  
);
```

```
CREATE TABLE bestellung (  
    id INT IDENTITY,  
    kunde_id INT,  
    datum DATE,  
    summe DECIMAL(10,2),  
    CONSTRAINT pk_bestellung  
        PRIMARY KEY (id),  
    CONSTRAINT fk_bestellung_kunde  
        FOREIGN KEY (kunde_id)  
        REFERENCES kunde (id)  
);
```

Daten einfügen

- Neue Datensätze in einer Tabelle können mit dem **INSERT**-Befehl erzeugt werden:

Syntax: **INSERT** [**INTO**] *<tabelle>*
 VALUES (*<werteliste>*) [, ...];

Beispiel: **INSERT INTO** kunde
 VALUES (1, 'Max', 'Mustermann', '19830417');

- Bei dieser Variante müssen für alle Spalten der Tabelle Werte in der korrekten Reihenfolge angegeben werden.
- Die übergebenen Werte müssen für die Spalten gültig sein.
- Es können auch mehrere Zeilen auf einmal eingefügt werden.

Daten einfügen

- Wenn nicht für alle Spalten Werte zur Verfügung stehen, kann eine Variante des **INSERT**-Befehls verwendet werden:

Syntax: **INSERT** [**INTO**] <table> (<spaltenliste>)
 VALUES (<werteliste>) [, (<werteliste>)];

Beispiel: **INSERT INTO** kunde (id, vorname, nachname)
 VALUES (2, 'Maria', 'Musterfrau');

- Bei dieser Variante müssen Werte für alle in der Spaltenliste angegebenen Spalten in der korrekten Reihenfolge angegeben werden.
- In den Spalten, die nicht angegeben wurden, müssen NULL-Werte erlaubt sein oder es muss einen DEFAULT-Wert geben (später).

Daten einfügen

- Eine Besonderheit gilt es bei IDENTITY-Spalten zu beachten:
Für eine IDENTITY-Spalte darf niemals ein Wert übergeben werden, da die Datenbank diese Werte selbst verwaltet.
- D.h., auch bei der **INSERT**-Variante ohne Spaltenliste darf kein Wert für die IDENTITY-Spalte angegeben werden:
Beispiel: **INSERT INTO** bestellung
VALUES (1, '20230829', 39.99);
- Dies ist der einzige Fall, bei dem die Anzahl der Werte nicht zur Anzahl der Spalten passt.

Daten aktualisieren

- Bestehende Daten werden mit dem **UPDATE**-Befehl verändert:

Syntax: **UPDATE** <table>
 SET <spalte> = <wert> [, ...]
 [**WHERE** <bedingung>];

Beispiel: **UPDATE** kunde
 SET vorname = 'Maximilian'
 WHERE id = 1;

- Der Befehl wäre auch ohne die **WHERE**-Klausel gültig, würde die Änderung dann aber bei allen Datensätzen durchführen.
- Es können auch Zuweisungen für mehrere Spalten gemacht werden, aber immer nur innerhalb einer Tabelle.

Daten löschen

- Bestehende Datensätze werden mit dem **DELETE**-Befehl entfernt:

Syntax: **DELETE FROM** *<tabelle>*
 [**WHERE** *<bedingung>*];

Beispiel: **DELETE FROM** kunde
 WHERE id = 2;

- Auch hier ist die **WHERE**-Klausel nicht unbedingt notwendig, ohne sie werden aber alle Datensätze gelöscht.
- Bei Tabellen mit **IDENTITY**-Spalten werden die gelöschten Werte nicht ersetzt, d.h. es bleiben Lücken in der Nummerierung.
- Eine Löschung kann nicht rückgängig gemacht werden.

Daten löschen

- Wenn alle Datensätze einer Tabelle gelöscht werden sollen, ist es günstiger den **TRUNCATE**-Befehl zu verwenden:

Syntax: **TRUNCATE TABLE** <table>;

Beispiel: **TRUNCATE TABLE** bestellung;

- Der **TRUNCATE**-Befehl kann von der Datenbank effizienter ausgeführt werden als der **DELETE**-Befehl, da hier weniger Informationen protokolliert werden.
- Hier gibt es aber keine **WHERE**-Klausel um die zu löschenden Datensätze einzuschränken. Es werden immer alle Datensätze der Tabelle gelöscht.

Daten löschen

Vergleich DELETE vs. TRUNCATE

DELETE	TRUNCATE
Eine Filterung der zu löschenden Datensätze kann mit der WHERE-Klausel erfolgen.	Eine Filterung der Datensätze ist nicht möglich, es werden immer alle Datensätze gelöscht.
Bei großen Datenmengen sehr langsam, da sehr viele Daten ins Transaktionsprotokoll geschrieben werden.	Auch bei großen Datenmengen schnell, da nur einige wenige Metadaten ins Transaktionsprotokoll geschrieben werden.
Eine evtl. vorhandene IDENTITY-Spalte wird nicht zurückgesetzt. Bei einem erneuten INSERT wird also einfach weiter gezählt.	Eine evtl. vorhandene IDENTITY-Spalte wird zurückgesetzt. Bei einem erneuten INSERT beginnen die Werte wieder am Anfang.
Ist auch möglich, wenn auf die Tabelle ein Fremdschlüssel verweist, sofern keine Fremdschlüsselwerte von DELETE betroffen sind.	Ist in Tabellen, auf die ein Fremdschlüssel verweist, grundsätzlich nicht möglich, selbst wenn der Fremdschlüssel deaktiviert ist.